

Rúbrica de evaluación: Actividades RA6

Criterio de evaluación	Peso	Sobresaliente (10-9)	Notable (8-7)	Suficiente (6-5)	Insuficiente (4-0)
1. Herramienta CASE y Sintaxis (CE i) <i>(PlantUML, exportación y entrega)</i>	20%	Código impecable, bien indentado y comentado. Dominio absoluto de las directivas de PlantUML. Entrega perfecta en .zip.	El código es limpio, compila sin errores y se ha aplicado el formato visual requerido (skinparam). Entrega correcta.	El código compila y genera la imagen, pero hay advertencias. La entrega cumple mínimamente el formato.	El código PlantUML no compila, da errores de sintaxis o no se han entregado los archivos en el formato/zip solicitado.
2. Identificación de Elementos UML (CE c, e, g) <i>(Nomenclatura y símbolos correctos)</i>	20%	Precisión técnica absoluta. Usa elementos avanzados correctamente (actores secundarios, transiciones reflexivas, focos de control).	Usa los elementos correctos del diagrama. Nomenclatura adecuada (verbos para acciones, adjetivos para estados).	Identifica la mayoría de los elementos, pero hay errores de concepto (ej. usa sustantivos en casos de uso en vez de verbos).	Confunde los elementos básicos (usa actores en lugar de estados, no pone nodos de inicio/fin, etc.). Nomenclatura errónea.
3. Lógica de Negocio y Flujo (CE d, f, h) <i>(Resolución del problema planteado)</i>	30%	Modela la realidad del enunciado a la perfección, sin lagunas lógicas, anticipando incluso posibles errores del sistema.	Resuelve el problema planteado casi en su totalidad. Las secuencias o actividades fluyen de forma lógica de principio a fin.	El flujo general tiene sentido, pero omite restricciones importantes del enunciado o hay "callejones sin salida".	El diagrama no resuelve el problema del enunciado. El flujo de datos es imposible o contradictorio.
4. Relaciones y Estructuras Complejas (CE d, f, h) <i>(Includes, Extends, Loops, Forks...)</i>	30%	Dominio total de la complejidad: herencia de actores perfecta, anidamiento de alt dentro de loop, concurrencia sin fisuras.	Usa correctamente estructuras complejas y de control, demostrando que entiende la concurrencia y las condiciones lógicas.	Intenta usar estructuras complejas, pero comete errores (ej. apunta el <<extend>> al revés, o el join no cuadra).	No utiliza o usa mal las relaciones avanzadas (<<include>>, bloques alt, barras fork).